

Algoritmos em Grafos

Busca em Largura e Busca em Profundidade

Conceitos, Simulações e Implementações em Java

BFS e DFS no estilo CLRS

Material Didático

Sumário

Introdução	2
Esquema de Cores para Vértices	2
Busca em Largura (BFS)	2
Ideia Geral	2
Aplicações da BFS	3
Simulação Passo a Passo da BFS	3
Pseudocódigo da BFS	6
Implementação Iterativa em Java (com Fila)	7
Implementação Recursiva em Java	8
Busca em Profundidade (DFS)	10
Ideia Geral	10
Aplicações da DFS	11
Simulação Passo a Passo da DFS	11
Pseudocódigo da DFS	14
Implementação Recursiva em Java	15
Implementação Iterativa em Java (com Pilha)	16
Comparação BFS × DFS	18
Referências	18

Introdução

Em muitos problemas computacionais é preciso visitar, de forma sistemática, todos os vértices de um grafo a partir de um vértice de partida, descobrindo quais são alcançáveis e em que ordem podem ser alcançados. As estratégias clássicas para essa tarefa são duas: a **busca em largura** (*Breadth-First Search*, ou BFS) e a **busca em profundidade** (*Depth-First Search*, ou DFS).

Embora ambas tenham o mesmo objetivo — percorrer um grafo de modo a visitar cada vértice alcançável a partir de uma origem — elas diferem radicalmente na *ordem* em que os vértices são descobertos:

- A **BFS** explora o grafo em *camadas*: primeiro o vértice de origem, depois todos os seus vizinhos diretos, depois os vizinhos dos vizinhos, e assim por diante. É natural usar uma estrutura de dados **fila** (FIFO) para guardar os vértices que ainda precisam ser processados.
- A **DFS** explora o grafo seguindo um caminho até o seu fim — isto é, até atingir um vértice cujos vizinhos já foram todos descobertos — e só então *retrocede* para explorar caminhos alternativos. A estrutura natural aqui é uma **pilha** (LIFO), seja ela explícita (versão iterativa) ou implícita por meio das chamadas recursivas (versão recursiva).

Esta apostila apresenta ambos os algoritmos seguindo o estilo do CLRS¹: começamos pela ideia intuitiva, em seguida apresentamos o pseudocódigo, depois ilustramos passo a passo a execução em um grafo concreto, e por fim discutimos duas implementações em Java de cada algoritmo — uma iterativa e outra recursiva.

Convenções. Trabalharemos com grafos não orientados, finitos e simples (sem laços nem arestas paralelas). Um grafo $G = (V, E)$ tem $|V| = n$ vértices e $|E| = m$ arestas. Salvo indicação contrária, supomos que G está representado por listas de adjacências.

Esquema de Cores para Vértices

Tanto a BFS quanto a DFS, na formulação do CLRS, classificam os vértices em três estados ao longo da execução. Esses estados são tradicionalmente representados por cores:

Cor	Significado
	Branco: vértice ainda não descoberto.
	Cinza: vértice descoberto, mas ainda em processamento.
	Preto: vértice já processado (todos os seus vizinhos foram examinados).

No início, todos os vértices são brancos. Ao serem *descobertos*, passam para cinza. Quando o algoritmo termina de processá-los, tornam-se pretos. Veremos que essa distinção em três cores é o que permite, por exemplo, evitar revisitas e detectar a estrutura da árvore de busca.

Busca em Largura (BFS)

Ideia Geral

Suponha que você esteja em uma cidade desconhecida e queira descobrir, a partir do ponto onde se encontra, quais bairros podem ser alcançados em $0, 1, 2, 3, \dots$ deslocamentos — onde cada “deslocamento” significa atravessar uma única rua. A BFS é exatamente esse processo: ela visita

¹Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 4.^a edição. MIT Press, 2022.

primeiro o vértice de partida (distância 0), depois todos os vértices à distância 1, em seguida todos à distância 2, e assim por diante.

Como consequência, a BFS calcula, para cada vértice alcançável, o **menor número de arestas** entre ele e a origem. Em grafos não ponderados, esse valor coincide com a distância do vértice à origem. A BFS também constrói naturalmente uma **árvore de busca em largura**: para cada vértice v descoberto, registra-se o vértice u a partir do qual v foi alcançado (denotado $\pi[v]$).

Aplicações da BFS

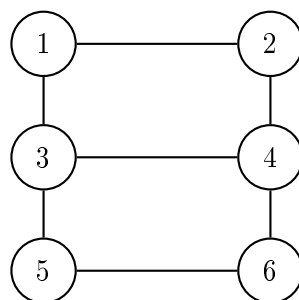
A busca em largura é a base de muitos algoritmos úteis:

- **Caminho mínimo em grafos não ponderados.** Encontrar o caminho com menor número de arestas entre dois vértices.
- **Verificação de conectividade.** Determinar se um grafo é conexo e quais são suas componentes conexas.
- **Verificação de bipartição.** Um grafo é bipartido se, e somente se, não contém ciclo de comprimento ímpar. A BFS pode colorir os vértices alternadamente nos níveis pares e ímpares e detectar uma violação.
- **Redes sociais.** Calcular o “grau de separação” entre duas pessoas em uma rede de amizades.
- **Robótica e jogos.** Encontrar a saída mais próxima em um labirinto modelado como grafo, onde cada célula é um vértice e células adjacentes são ligadas por arestas.
- **Web crawling.** Descobrir páginas web partindo de uma página inicial, expandindo nível a nível pelos hiperlinks.
- **Captura em jogos de tabuleiro.** Em jogos como Go, identificar grupos de pedras conectadas para verificar capturas.

Simulação Passo a Passo da BFS

Vamos simular a BFS sobre o grafo G a seguir, partindo do vértice 1. Suponha que, para cada vértice, a lista de adjacências esteja em ordem crescente.

Grafo de entrada.



Listas de adjacências:

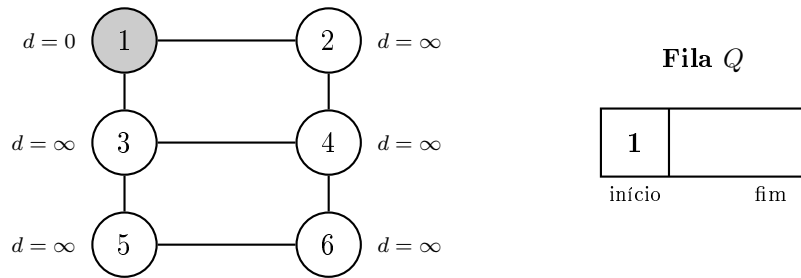
$$Adj[1] = (2, 3), \quad Adj[2] = (1, 4), \quad Adj[3] = (1, 4, 5),$$

$$Adj[4] = (2, 3, 6), \quad Adj[5] = (3, 6), \quad Adj[6] = (4, 5).$$

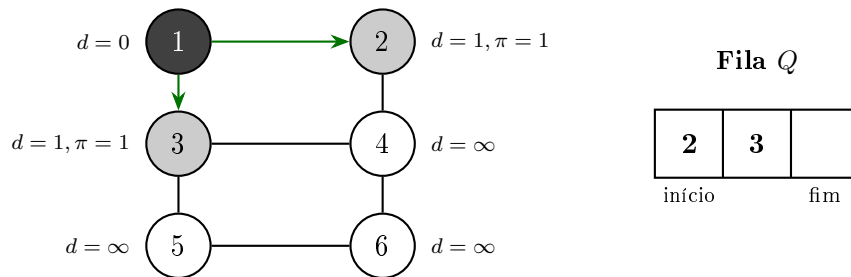
Como ler cada passo. Cada passo apresenta:

- o vértice u acabou de ser desenfileirado (ou, no passo inicial, o vértice de origem);
- o estado das cores e dos valores $d[v]$ (distância) e $\pi[v]$ (predecessor) de cada vértice;
- o conteúdo atual da fila Q ao final do passo.

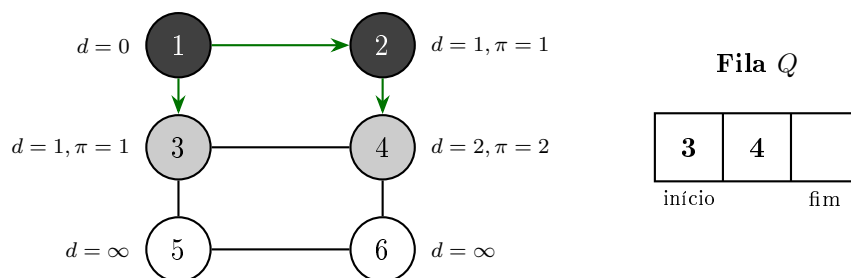
Passo 0 — Inicialização. Marcamos o vértice 1 como cinza, com $d[1] = 0$ e $\pi[1] = \text{nulo}$. Os demais começam brancos com $d[v] = \infty$ e $\pi[v] = \text{nulo}$. Enfileiramos o vértice 1.



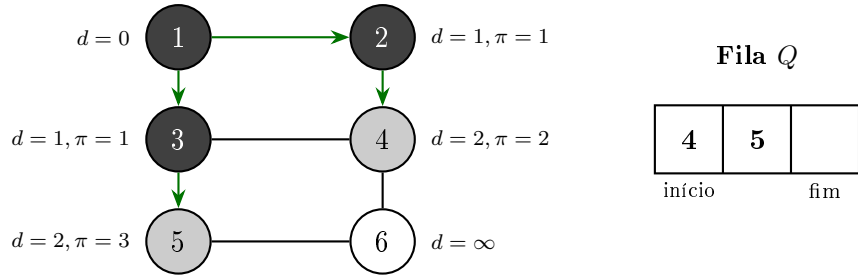
Passo 1 — Processa $u = 1$. Desenfileiramos 1. Examinamos seus vizinhos: 2 (branco) e 3 (branco). Ambos são descobertos: ficam cinzas, recebem $d = 1$ e $\pi = 1$, e entram na fila. Em seguida, 1 fica preto.



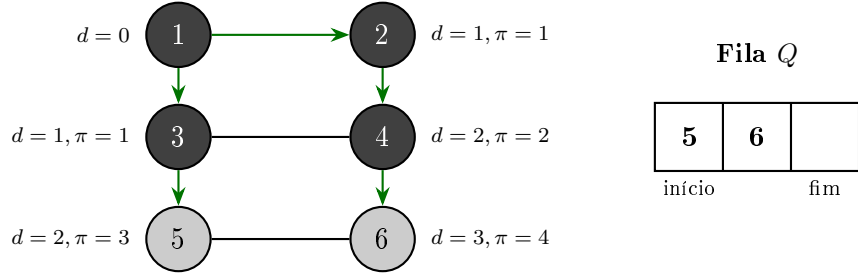
Passo 2 — Processa $u = 2$. Desenfileiramos 2. Examinamos seus vizinhos: 1 (preto, ignora) e 4 (branco). 4 é descoberto: cinza, $d = 2$, $\pi = 2$, entra na fila. 2 fica preto.



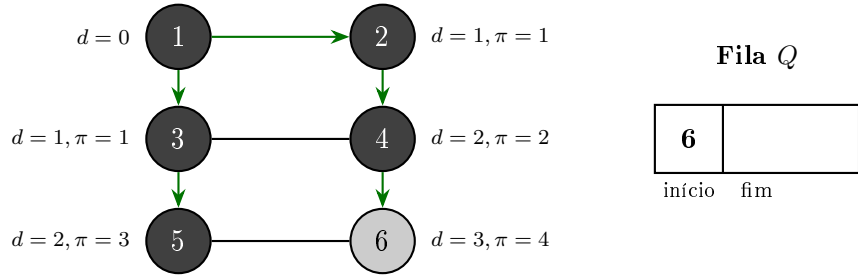
Passo 3 — Processa $u = 3$. Desenfileiramos 3. Examinamos seus vizinhos: 1 (preto, ignora), 4 (cinza, ignora) e 5 (branco). 5 é descoberto: cinza, $d = 2$, $\pi = 3$, entra na fila. 3 fica preto.



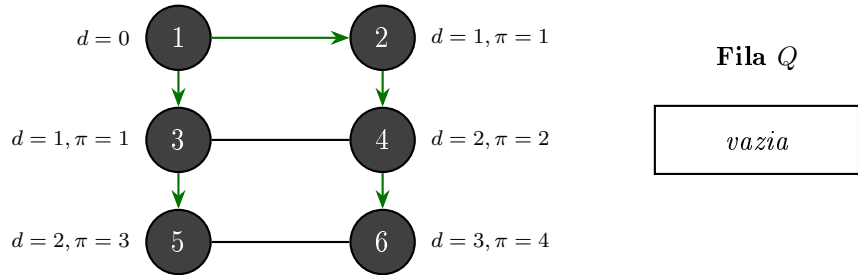
Passo 4 — Processa $u = 4$. Desenfileiramos 4. Examinamos seus vizinhos: 2 (preto), 3 (preto) e 6 (branco). 6 é descoberto: cinza, $d = 3$, $\pi = 4$, entra na fila. 4 fica preto.



Passo 5 — Processa $u = 5$. Desenfileiramos 5. Seus vizinhos são 3 (preto) e 6 (cinza). Nenhum é branco — ninguém novo entra na fila. 5 fica preto.



Passo 6 — Processa $u = 6$ (último). Desenfileiramos 6. Seus vizinhos 4 e 5 são pretos. 6 fica preto. A fila está vazia: a BFS termina.



Resultado. As arestas verdes desenhadas formam a *árvore de busca em largura* enraizada em 1. As distâncias $d[v]$ correspondem exatamente ao número de arestas no caminho mais curto de 1 até v no grafo original.

v	1	2	3	4	5	6
$d[v]$	0	1	1	2	2	3
$\pi[v]$	—	1	1	2	3	4

Pseudocódigo da BFS

O algoritmo abaixo recebe um grafo G e um vértice de origem $s \in V(G)$. Ao final, para cada vértice v alcançável a partir de s , $d[v]$ contém a distância (em arestas) de s a v , e $\pi[v]$ contém o predecessor de v na árvore de busca em largura.

Algoritmo 1: BFS(G, s)

```

1 para cada  $u \in V(G) \setminus \{s\}$  faça
2    $cor[u] \leftarrow \text{branco}$ 
3    $d[u] \leftarrow \infty$ 
4    $\pi[u] \leftarrow \text{nulo}$ 
5 fim
6  $cor[s] \leftarrow \text{cinza}$ 
7  $d[s] \leftarrow 0$ 
8  $\pi[s] \leftarrow \text{nulo}$ 
9  $Q \leftarrow \emptyset$  // fila vazia
10 ENFILEIRA( $Q, s$ )
11 enquanto  $Q \neq \emptyset$  faça
12    $u \leftarrow \text{DESENFILEIRA}(Q)$ 
13   para cada  $v \in Adj[u]$  faça
14     se  $cor[v] = \text{branco}$  então
15        $cor[v] \leftarrow \text{cinza}$ 
16        $d[v] \leftarrow d[u] + 1$ 
17        $\pi[v] \leftarrow u$ 
18       ENFILEIRA( $Q, v$ )
19   fim
20 fim
21  $cor[u] \leftarrow \text{preto}$ 
22 fim

```

Análise de Complexidade Linha a Linha. A tabela abaixo dá o custo total acumulado de cada linha em uma execução completa da BFS. Linhas estruturais (**fim** de bloco) não contribuem para o tempo e estão marcadas com “—”. A análise assume que o grafo é representado por listas de adjacências.

Linha	Pseudocódigo	Custo total
1	para cada $u \in V(G) \setminus \{s\}$ faça (teste)	$\Theta(V)$
2	$cor[u] \leftarrow \text{branco}$	$\Theta(V)$
3	$d[u] \leftarrow \infty$	$\Theta(V)$
4	$\pi[u] \leftarrow \text{nulo}$	$\Theta(V)$
5	fim	—
6	$cor[s] \leftarrow \text{cinza}$	$\Theta(1)$
7	$d[s] \leftarrow 0$	$\Theta(1)$
8	$\pi[s] \leftarrow \text{nulo}$	$\Theta(1)$
9	$Q \leftarrow \emptyset$ (cria fila vazia)	$\Theta(1)$
10	ENFILEIRA(Q, s)	$\Theta(1)$
11	enquanto $Q \neq \emptyset$ faça (teste, $ V + 1$ vezes)	$\Theta(V)$
12	$u \leftarrow \text{DESENFILEIRA}(Q)$ (cada vértice 1 vez)	$\Theta(V)$
13	para cada $v \in Adj[u]$ faça (varredura agregada [†])	$\Theta(V + E)$
14	se $cor[v] = \text{branco}$ então	$\Theta(E)$
15	$cor[v] \leftarrow \text{cinza}$ (descoberta, $ V - 1$ vezes)	$\Theta(V)$
16	$d[v] \leftarrow d[u] + 1$	$\Theta(V)$
17	$\pi[v] \leftarrow u$	$\Theta(V)$

continua na próxima página

Linha	Pseudocódigo	Custo total
18	ENFILEIRA(Q, v)	$\Theta(V)$
19	fm	—
20	fm	—
21	$cor[u] \leftarrow preto$ (cada vértice 1 vez)	$\Theta(V)$
22	fm	—
Total:		$\Theta(V + E)$

[†]O laço **para cada** da linha 13 é percorrido em todas as iterações do **enquanto**: cada vértice u contribui com $|Adj[u]|$ iterações. Somando sobre todos os vértices, o número total de iterações é $\sum_{u \in V} |Adj[u]| = 2|E|$ (cada aresta é contada nas duas extremidades), de onde sai o termo $\Theta(|E|)$. A constante de inicialização do laço em cada chamada contribui com $\Theta(|V|)$ adicional, totalizando $\Theta(|V| + |E|)$.

Implementação Iterativa em Java (com Fila)

A implementação direta do pseudocódigo usa uma `Queue` da biblioteca padrão do Java (uma `LinkedList` serve como fila FIFO eficiente).

```

1  import java.util.*;
2
3  public class BFSIterativa {
4
5      // Lista de adjacências: vizinhos[u] = lista de vizinhos de u
6      private final List<List<Integer>> vizinhos;
7      private final int n;
8
9      public BFSIterativa(int n) {
10         this.n = n;
11         this.vizinhos = new ArrayList<>(n);
12         for (int i = 0; i < n; i++) {
13             vizinhos.add(new ArrayList<>());
14         }
15     }
16
17     public void adicionarAresta(int u, int v) {
18         vizinhos.get(u).add(v);
19         vizinhos.get(v).add(u);
20     }
21
22     /**
23      * Executa BFS a partir de s. Preenche os arrays d e pi.
24      * Estados: 0 = branco, 1 = cinza, 2 = preto.
25      */
26     public void bfs(int s, int[] d, int[] pi) {
27         int[] cor = new int[n];
28         for (int u = 0; u < n; u++) {
29             cor[u] = 0;
30             d[u] = Integer.MAX_VALUE;
31             pi[u] = -1;
32         }
33         cor[s] = 1;
34         d[s] = 0;
35
36         Queue<Integer> fila = new LinkedList<>();

```



```

37     fila.add(s);
38
39     while (!fila.isEmpty()) {
40         int u = fila.poll();
41         for (int v : vizinhos.get(u)) {
42             if (cor[v] == 0) {
43                 cor[v] = 1;
44                 d[v] = d[u] + 1;
45                 pi[v] = u;
46                 fila.add(v);
47             }
48         }
49         cor[u] = 2;
50     }
51 }
52
53 public static void main(String[] args) {
54     BFSIterativa g = new BFSIterativa(7); // indices 0..6, usamos
55     1..6
56     g.adicionarAresta(1, 2);
57     g.adicionarAresta(1, 3);
58     g.adicionarAresta(2, 4);
59     g.adicionarAresta(3, 4);
60     g.adicionarAresta(3, 5);
61     g.adicionarAresta(4, 6);
62     g.adicionarAresta(5, 6);
63
64     int[] d = new int[7];
65     int[] pi = new int[7];
66     g.bfs(1, d, pi);
67
68     System.out.println("v    d[v]    pi[v]");
69     for (int v = 1; v <= 6; v++) {
70         System.out.printf("%d    %2d    %2d\n", v, d[v], pi[v]);
71     }
72 }

```

Código 1: BFS iterativa com fila (Java)

v	d[v]	pi[v]
1	0	-1
2	1	1
3	1	1
4	2	2
5	2	3
6	3	4

Implementação Recursiva em Java

A BFS não é, por natureza, um algoritmo recursivo — o uso natural da recursão (com a pilha de chamadas) leva à profundidade, não à largura. Mas é possível escrevê-la de forma recursiva delegando a fila como parâmetro, recursão de cauda processando um vértice por chamada.

```

1 import java.util.*;
2
3 public class BFSRecurativa {
4
5     private final List<List<Integer>> vizinhos;
6     private final int n;
7
8     public BFSRecurativa(int n) {
9         this.n = n;
10        this.vizinhos = new ArrayList<>(n);
11        for (int i = 0; i < n; i++) {
12            vizinhos.add(new ArrayList<>());
13        }
14    }
15
16    public void adicionarAresta(int u, int v) {
17        vizinhos.get(u).add(v);
18        vizinhos.get(v).add(u);
19    }
20
21    /**
22     * Inicializa as estruturas e dispara a recursao.
23     */
24    public void bfs(int s, int[] d, int[] pi) {
25        int[] cor = new int[n];
26        for (int u = 0; u < n; u++) {
27            cor[u] = 0;
28            d[u] = Integer.MAX_VALUE;
29            pi[u] = -1;
30        }
31        cor[s] = 1;
32        d[s] = 0;
33
34        Queue<Integer> fila = new LinkedList<>();
35        fila.add(s);
36
37        bfsRec(fila, cor, d, pi);
38    }
39
40    /**
41     * Cada chamada recursiva processa exatamente um vertice
42     * (o que esta na frente da fila) e entao recorre.
43     */
44    private void bfsRec(Queue<Integer> fila, int[] cor, int[] d, int[]
        pi) {
45        if (fila.isEmpty()) {
46            return; // caso base: nada mais a processar
47        }
48        int u = fila.poll();
49        for (int v : vizinhos.get(u)) {
50            if (cor[v] == 0) {
51                cor[v] = 1;
52                d[v] = d[u] + 1;
53                pi[v] = u;
54                fila.add(v);
55            }
56        }
57        cor[u] = 2;

```

```

58     bfsRec(fila, cor, d, pi);
59 }
60
61 public static void main(String[] args) {
62     BFSRecursiva g = new BFSRecursiva(7);
63     g.adicionarAresta(1, 2);
64     g.adicionarAresta(1, 3);
65     g.adicionarAresta(2, 4);
66     g.adicionarAresta(3, 4);
67     g.adicionarAresta(3, 5);
68     g.adicionarAresta(4, 6);
69     g.adicionarAresta(5, 6);
70
71     int[] d = new int[7];
72     int[] pi = new int[7];
73     g.bfs(1, d, pi);
74
75     System.out.println("v    d[v]    pi[v]");
76     for (int v = 1; v <= 6; v++) {
77         System.out.printf("%d    %2d    %2d\n", v, d[v], pi[v]);
78     }
79 }
80 }

```

Código 2: BFS recursiva passando a fila como parâmetro (Java)

Observação. Embora a recursão seja válida, ela transforma a pilha de chamadas em um *contador* de iterações: cada nível da recursão corresponde a um vértice desenfileirado. Em grafos grandes, isso pode causar estouro de pilha (`StackOverflowError`) — a versão iterativa é, na prática, sempre preferível para a BFS.

Busca em Profundidade (DFS)

Ideia Geral

Imagine que você está explorando um labirinto com uma lanterna na mão e um carretel de barbante. Ao entrar em um corredor, você desenrola barbante. Toda vez que chega em uma bifurcação, escolhe um corredor não explorado e segue em frente. Quando atinge um beco sem saída — ou um corredor onde todos os caminhos partindo dali já foram percorridos — você *volta* seguindo o barbante (*backtracking*) até a bifurcação anterior, e tenta um corredor diferente.

Esse é exatamente o comportamento da **busca em profundidade**: ela vai o mais fundo possível seguindo um caminho antes de retroceder. Em termos algorítmicos:

- Ao descobrir um vértice u , marcamos u como cinza.
- Examinamos cada vizinho v de u ; se v for branco, fazemos uma chamada recursiva para visitar v .
- Quando todos os vizinhos de u tiverem sido examinados, marcamos u como preto e voltamos ao chamador.

Diferente da BFS, a DFS não calcula distâncias mínimas em grafos não ponderados. Em compensação, ela produz informações estruturais muito ricas sobre o grafo — como tempos de descoberta

e finalização, a árvore de busca em profundidade e a classificação de arestas — que são a base de muitos algoritmos clássicos.

Aplicações da DFS

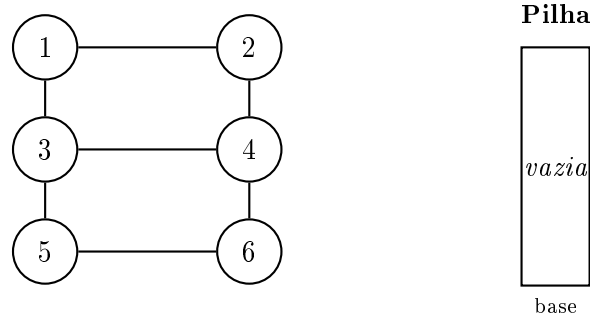
A busca em profundidade é a base de algoritmos como:

- **Detecção de ciclos.** Encontrar ciclos em um grafo (orientado ou não).
- **Ordenação topológica.** Ordenar tarefas com dependências (DAGs).
- **Componentes fortemente conexas** em grafos orientados (algoritmo de Tarjan e algoritmo de Kosaraju).
- **Pontes e pontos de articulação.** Identificar arestas e vértices cuja remoção desconecta o grafo.
- **Resolução de labirintos e quebra-cabeças.** Explorar caminhos com retrocesso (*backtracking*).
- **Análise sintática.** Percorrer árvores de derivação em compiladores e interpretadores.
- **Geração de labirintos.** Algoritmos como o *recursive backtracker* criam labirintos via DFS aleatorizada.
- **Travessia de árvores** (pré-ordem, em-ordem e pós-ordem são casos particulares de DFS).

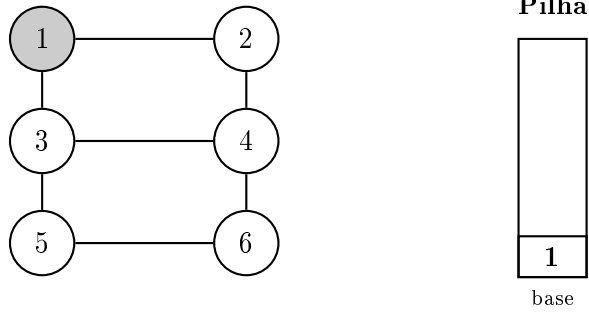
Simulação Passo a Passo da DFS

Vamos simular a DFS sobre o mesmo grafo da BFS, partindo do vértice 1. As listas de adjacências estão em ordem crescente.

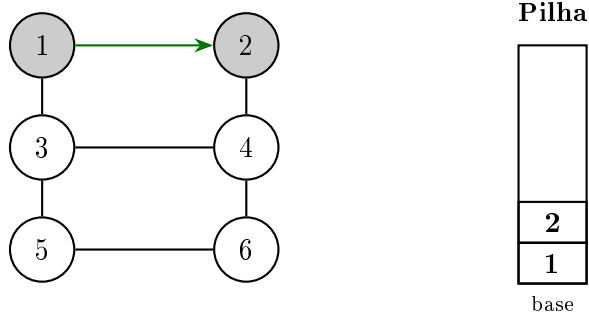
Passo 0 — Inicialização. Todos os vértices brancos. Pilha de chamadas vazia. Chamamos $\text{DFS-VISITA}(1)$.



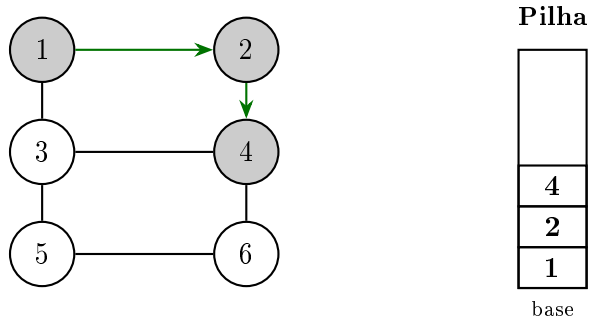
Passo 1 — $\text{DFS-VISITA}(1)$. 1 vira cinza. Examina vizinhos em ordem: 2 é branco — chamamos $\text{DFS-VISITA}(2)$.



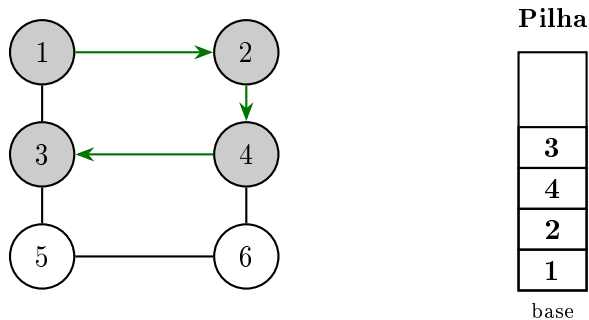
Passo 2 — DFS-VISITA(2). 2 vira cinza, $\pi[2] = 1$. Examina vizinhos: 1 (cinza, ignora), 4 (branco) — chamamos DFS-VISITA(4).



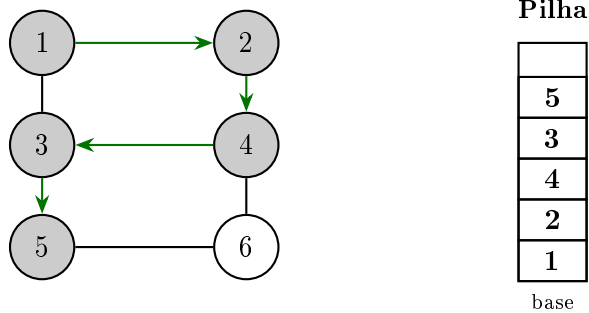
Passo 3 — DFS-VISITA(4). 4 vira cinza, $\pi[4] = 2$. Examina vizinhos: 2 (cinza), 3 (branco) — chamamos DFS-VISITA(3).



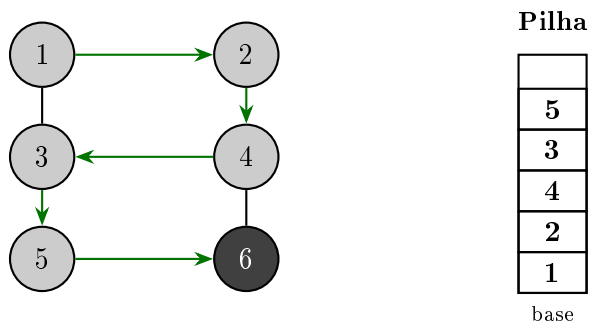
Passo 4 — DFS-VISITA(3). 3 vira cinza, $\pi[3] = 4$. Examina vizinhos: 1 (cinza), 4 (cinza), 5 (branco) — chamamos DFS-VISITA(5).



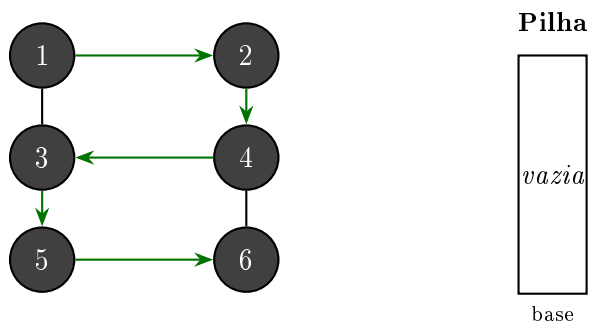
Passo 5 — DFS-VISITA(5). 5 vira cinza, $\pi[5] = 3$. Examina vizinhos: 3 (cinza), 6 (branco) — chamamos DFS-VISITA(6).



Passo 6 — DFS-VISITA(6). 6 vira cinza, $\pi[6] = 5$. Examina vizinhos: 4 (cinza), 5 (cinza). Nenhum branco. 6 fica preto e retornamos.



Passo 7 — Retorno em cascata. Voltamos para 5: já examinou todos os vizinhos $\Rightarrow$ 5 preto, retorna. Voltamos para 3: já examinou todos $\Rightarrow$ 3 preto, retorna. Voltamos para 4: falta examinar 6 (já preto, ignora) $\Rightarrow$ 4 preto, retorna. Voltamos para 2: já examinou todos $\Rightarrow$ 2 preto, retorna. Voltamos para 1: falta examinar 3 (já preto, ignora) $\Rightarrow$ 1 preto.



Resultado. A árvore de busca em profundidade tem o formato de uma cadeia $1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 5 \rightarrow 6$. Compare-a com a árvore da BFS, que tem o aspecto de uma árvore mais “larga”. Esse contraste reflete fielmente a diferença filosófica entre os dois algoritmos.

v	1	2	3	4	5	6
$\pi[v]$	—	1	4	2	3	5

Pseudocódigo da DFS

Apresentamos a versão recursiva clássica do CLRS. Ela percorre todos os vértices do grafo (não apenas os alcançáveis a partir de uma origem fixa), pois pode haver várias componentes conexas.

Algoritmo 2: DFS(G)

```

1 para cada  $u \in V(G)$  faça
2   |  $cor[u] \leftarrow branco$ 
3   |  $\pi[u] \leftarrow nulo$ 
4 fim
5 para cada  $u \in V(G)$  faça
6   | se  $cor[u] = branco$  então
7   |   | DFS-VISITA( $G, u$ )
8   | fim
9 fim

```

Algoritmo 3: DFS-Visita(G, u)

```

1  $cor[u] \leftarrow cinza$  //  $u$  foi descoberto
2 para cada  $v \in Adj[u]$  faça
3   | se  $cor[v] = branco$  então
4   |   |  $\pi[v] \leftarrow u$ 
5   |   | DFS-VISITA( $G, v$ )
6   | fim
7 fim
8  $cor[u] \leftarrow preto$  //  $u$  está terminado

```

Análise de Complexidade Linha a Linha. A análise abrange os dois algoritmos. O método DFS-VISITA é invocado exatamente uma vez para cada vértice (totalizando $|V|$ chamadas, somando todas as componentes conexas). Linhas estruturais estão marcadas com “—”. A análise assume listas de adjacências.

Linha	Pseudocódigo	Custo total
<i>Algoritmo 2 — DFS(G) (executado 1 vez)</i>		
1	para cada $u \in V(G)$ faça (teste)	$\Theta(V)$
2	$cor[u] \leftarrow branco$	$\Theta(V)$
3	$\pi[u] \leftarrow nulo$	$\Theta(V)$
4	fim	—
5	para cada $u \in V(G)$ faça (teste)	$\Theta(V)$
6	se $cor[u] = branco$ então	$\Theta(V)$
7	DFS-VISITA(G, u) (dispara recursão)	(ver abaixo)
8	fim	—
9	fim	—
<i>Algoritmo 3 — DFS-VISITA(G, u) (V chamadas, somando todas)</i>		
1	$cor[u] \leftarrow cinza$ (cada vértice 1 vez)	$\Theta(V)$
2	para cada $v \in Adj[u]$ faça (varredura agregada [†])	$\Theta(V + E)$
3	se $cor[v] = branco$ então	$\Theta(E)$
4	$\pi[v] \leftarrow u$ ($ V - 1$ vezes)	$\Theta(V)$
5	DFS-VISITA(G, v) (cascata)	(recursão)
6	fim	—
7	fim	—
8	$cor[u] \leftarrow preto$ (cada vértice 1 vez)	$\Theta(V)$
Total:		$\Theta(V + E)$

[†]Cada chamada de $\text{DFS-VISITA}(G, u)$ percorre a lista $\text{Adj}[u]$. Como DFS-VISITA é invocada uma única vez por vértice e $\sum_{u \in V} |\text{Adj}[u]| = 2|E|$, o trabalho total da linha 2 somado sobre todas as chamadas é $\Theta(|V| + |E|)$ (o termo $\Theta(|V|)$ contabiliza a inicialização do laço em cada chamada).

Implementação Recursiva em Java

A versão recursiva é a tradução direta do pseudocódigo: a pilha de chamadas faz, de forma transparente, o papel da pilha do algoritmo.

```

1  import java.util.*;
2
3  public class DFSRecursiva {
4
5      private final List<List<Integer>> vizinhos;
6      private final int n;
7      private int[] cor;
8      private int[] pi;
9
10     public DFSRecursiva(int n) {
11         this.n = n;
12         this.vizinhos = new ArrayList<>(n);
13         for (int i = 0; i < n; i++) {
14             vizinhos.add(new ArrayList<>());
15         }
16     }
17
18     public void adicionarAresta(int u, int v) {
19         vizinhos.get(u).add(v);
20         vizinhos.get(v).add(u);
21     }
22
23     /**
24      * Executa DFS a partir de s.
25      * Estados: 0 = branco, 1 = cinza, 2 = preto.
26      */
27     public int[] dfs(int s) {
28         cor = new int[n];
29         pi = new int[n];
30         for (int u = 0; u < n; u++) {
31             cor[u] = 0;
32             pi[u] = -1;
33         }
34         dfsVisita(s);
35         return pi;
36     }
37
38     private void dfsVisita(int u) {
39         cor[u] = 1;
40         for (int v : vizinhos.get(u)) {
41             if (cor[v] == 0) {
42                 pi[v] = u;
43                 dfsVisita(v);
44             }
45         }
46         cor[u] = 2;

```



```

47     }
48
49     public static void main(String[] args) {
50         DFSRecurativa g = new DFSRecurativa(7);
51         g.adicionarAresta(1, 2);
52         g.adicionarAresta(1, 3);
53         g.adicionarAresta(2, 4);
54         g.adicionarAresta(3, 4);
55         g.adicionarAresta(3, 5);
56         g.adicionarAresta(4, 6);
57         g.adicionarAresta(5, 6);
58
59         int[] pi = g.dfs(1);
60
61         System.out.println("v    pi[v]");
62         for (int v = 1; v <= 6; v++) {
63             System.out.printf("%d    %2d%n", v, pi[v]);
64         }
65     }
66 }

```

Código 3: DFS recursiva (Java)

Implementação Iterativa em Java (com Pilha)

Para evitar o uso da pilha de chamadas (e o risco de `StackOverflowError` em grafos profundos), substituímos a recursão por uma pilha explícita (`Deque<Integer>`, com `push` e `pop`).

Um ponto sutil: para que a versão iterativa produza *exatamente a mesma* árvore que a versão recursiva, é preciso inserir os vizinhos na pilha em ordem *inversa* (pois a pilha é LIFO, o último a entrar é o primeiro a sair).

```

1  import java.util.*;
2
3  public class DFSIterativa {
4
5      private final List<List<Integer>> vizinhos;
6      private final int n;
7
8      public DFSIterativa(int n) {
9          this.n = n;
10         this.vizinhos = new ArrayList<>(n);
11         for (int i = 0; i < n; i++) {
12             vizinhos.add(new ArrayList<>());
13         }
14     }
15
16     public void adicionarAresta(int u, int v) {
17         vizinhos.get(u).add(v);
18         vizinhos.get(v).add(u);
19     }
20
21     /**
22      * Executa DFS iterativa a partir de s usando pilha explícita.
23      * Estados: 0 = branco, 1 = cinza, 2 = preto.
24      */
25     public int[] dfs(int s) {

```

```

26     int[] cor = new int[n];
27     int[] pi = new int[n];
28     for (int u = 0; u < n; u++) {
29         cor[u] = 0;
30         pi[u] = -1;
31     }
32
33     Deque<Integer> pilha = new ArrayDeque<>();
34     pilha.push(s);
35     cor[s] = 1;
36
37     while (!pilha.isEmpty()) {
38         int u = pilha.peek();
39         int proximoBranco = -1;
40
41         // Procura o proximo vizinho branco de u
42         for (int v : vizinhos.get(u)) {
43             if (cor[v] == 0) {
44                 proximoBranco = v;
45                 break;
46             }
47         }
48
49         if (proximoBranco != -1) {
50             // Avanca: descobre o vizinho branco
51             cor[proximoBranco] = 1;
52             pi[proximoBranco] = u;
53             pilha.push(proximoBranco);
54         } else {
55             // Backtrack: todos os vizinhos ja foram tratados
56             cor[u] = 2;
57             pilha.pop();
58         }
59     }
60     return pi;
61 }
62
63 public static void main(String[] args) {
64     DFSIterativa g = new DFSIterativa(7);
65     g.adicionarAresta(1, 2);
66     g.adicionarAresta(1, 3);
67     g.adicionarAresta(2, 4);
68     g.adicionarAresta(3, 4);
69     g.adicionarAresta(3, 5);
70     g.adicionarAresta(4, 6);
71     g.adicionarAresta(5, 6);
72
73     int[] pi = g.dfs(1);
74
75     System.out.println("v    pi[v]");
76     for (int v = 1; v <= 6; v++) {
77         System.out.printf("%d    %2d%n", v, pi[v]);
78     }
79 }
80 }

```

Código 4: DFS iterativa com pilha (Java)

Observação. Existe uma variante mais simples (porém um pouco menos fiel à recursão) na qual, ao desempilhar u , empilhamos todos os seus vizinhos brancos de uma vez. Essa versão consome um pouco mais de memória e produz uma ordem de descoberta levemente diferente, mas continua sendo uma DFS legítima.

Comparação BFS $\times$ DFS

Sintetizamos abaixo as principais diferenças entre os dois algoritmos.

	BFS	DFS
Estrutura natural	Fila (FIFO)	Pilha (LIFO ou recursão)
Ordem de visita	Por camadas (níveis)	Em profundidade primeiro
Distâncias mínimas	Calcula em grafos não ponderados	Não
Árvore resultante	Tende a ser larga e rasa	Tende a ser estreita e profunda
Memória usada	Pode chegar a $\Theta(V)$ na fila (todos de um nível)	Profundidade máxima da recursão
Aplicações típicas	Caminho mínimo, componentes conexas, bipartição	Ciclos, ordenação topológica, componentes fortemente conexas, pontes, geração e resolução de labirintos
Tempo de execução	$\Theta(V + E)$	$\Theta(V + E)$

Ambas têm a mesma complexidade assintótica de tempo — visitam cada vértice e cada aresta uma quantidade constante de vezes — e ambas são lineares em $|V| + |E|$. A escolha entre elas depende do problema:

- Se for preciso **distância mínima** (em arestas), use BFS.
- Se for preciso **explorar caminhos** um por vez, encontrar ciclos, fazer ordenação topológica, descobrir componentes fortemente conexas ou simular *backtracking*, use DFS.

Referências

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. *Introduction to Algorithms*. 4.^a edição. MIT Press, 2022. Capítulo 20: Elementary Graph Algorithms.

BONDY, J. A.; MURTY, U. S. R. *Graph Theory*. Nova York: Springer, 2008. (Graduate Texts in Mathematics, v. 244).

FEOFILOFF, Paulo. *Algoritmos para Grafos*. Disponível em: https://www.ime.usp.br/~pf/algoritmos_para_grafos/. Acesso em: 2026.